

Relazione - Versione 3: Fondamenti del training

Introduzione

La Versione 3 riorganizza il modello neurale a caratteri in NumPy della Versione 2 trasformandolo in un esperimento di training strutturato.

Corpus, vocabolario, compito basato su caratteri adiacenti, rappresentazione numerica, input one-hot, matrice dei pesi 27 per 27, probabilità softmax, obiettivo cross-entropy e generatore autoregressivo restano invariati. Il nuovo concetto è il processo di training: suddivisione riproducibile dei dati, mini-batch mescolati e monitoraggio sistematico della validation.

```
Caratteri
↓
Identificatori interi
↓
Split training / validation
↓
Mini-batch mescolati
↓
Aggiornamenti dei pesi
↓
Monitoraggio validation
↓
Testo generato
```

Obiettivo della Versione 3

Il modello deve essere in grado di:

- riutilizzare corpus, vocabolario, codifica numerica e architettura neural bigram della Versione 2;
- dividere in modo riproducibile i 439 esempi input-target adiacenti in training e validation;
- mescolare soltanto gli esempi di training all'inizio di ogni epoca;
- aggiornare i pesi tramite mini-batch gradient descent;
- misurare training loss e validation loss dopo ogni epoca;
- individuare l'epoca con validation loss minima senza usare gli esempi di validation per aggiornare i pesi;
- riprodurre inizializzazione, split dei dati, training e generazione tramite seed locali espliciti.

Architettura

Il flusso completo della Versione 3 è:

```
Corpus: 440 caratteri
↓
Vocabolario: 27 caratteri
↓
439 coppie input-target adiacenti
↓
Matrice di input one-hot: (439, 27)
↓
Split riproducibile 80 / 20 degli esempi
↓
Training: 351 | Validation: 88
↓
Aggiornamenti con mini-batch mescolati
↓
Monitoraggio training e validation loss
↓
Generatore autoregressivo
```

Il contesto contiene ancora un solo carattere. La Versione 3 resta quindi lo stesso modello neural bigram a caratteri introdotto nella Versione 2.

Split training e validation

Un generatore NumPy locale con seed 42 crea una permutazione casuale dei 439 indici degli esempi. Il primo 80 per cento viene assegnato al training e gli esempi rimanenti alla validation.

```
split_position = int(439 * 0.8) = 351
esempi di training = 351
esempi di validation = 88
```

I due sottoinsiemi sono disgiunti per indice e insieme contengono tutti i 439 esempi. Gli esempi di validation vengono usati soltanto per misurare la loss e non aggiornano mai i pesi.

Poiché lo split opera sui singoli esempi formati da caratteri adiacenti, lo stesso tipo di bigramma può comparire in entrambi i sottoinsiemi. La validation mantiene quindi separati esempi o posizioni, non necessariamente transizioni di caratteri mai viste prima.

Modello numerico invariato

Ogni carattere di input continua a essere rappresentato da un vettore one-hot di lunghezza 27. Il modello contiene ancora una sola matrice di pesi addestrabile 27 per 27.

Forma matrice input: (439, 27)
Forma matrice pesi: (27, 27)
Parametri addestrabili: 729

I pesi iniziali sono piccoli valori casuali generati con un seed NumPy locale pari a 42, mantenendo la stessa parametrizzazione trasparente della Versione 2.

Mini-batch e shuffling

All'inizio di ogni epoca di training, i 351 esempi di training vengono mescolati con un generatore casuale locale dedicato. Gli esempi mescolati vengono poi elaborati in mini-batch di dimensione 32.

351 esempi di training
↓
10 mini-batch completi da 32
+
1 mini-batch finale da 31
↓
11 aggiornamenti dei pesi per epoca

Con 100 epoche di training, il modello esegue 1.100 aggiornamenti mini-batch dei parametri. Il validation set non viene mescolato per l'apprendimento perché non partecipa mai all'ottimizzazione.

Riproducibilità sistematica

Lo stesso valore di seed, 42, viene usato da generatori casuali locali separati per inizializzazione dei pesi, split dei dati, shuffling del training e dimostrazione di generazione. I generatori sono oggetti indipendenti, quindi il notebook non dipende dallo stato casuale globale di NumPy.

Obiettivo di training

Per ogni mini-batch, il modello applica gli stessi calcoli introdotti nella Versione 2:

- moltiplica gli input one-hot per la matrice dei pesi ottenendo i logits;
- applica una softmax numericamente stabile per ottenere le probabilità del carattere successivo;
- misura la cross-entropy rispetto agli ID target corretti;
- calcola direttamente in NumPy i gradienti dei logits e dei pesi;
- aggiorna i pesi tramite gradient descent.

```
logits_gradient = probabilities - targets
logits_gradient /= batch_size
weights_gradient = batch_inputs.T @ logits_gradient
weights -= learning_rate * weights_gradient
```

Configurazione del training:

```
Learning rate: 1.0
Batch size: 32
Epoche: 100
Seed: 42
```

Risultato del training

```
Training loss iniziale: 3.297216
Training loss finale: 1.987639
Validation loss iniziale: 3.297493
Validation loss finale: 2.903679
Validation loss migliore: 2.893800
Epoca migliore: 64
```

La training loss diminuisce per tutta l'esecuzione. La validation loss inizialmente diminuisce, raggiunge il minimo all'epoca 64 e poi aumenta leggermente. Questo è possibile perché viene ottimizzata direttamente soltanto la training loss.

La Versione 3 registra l'epoca con validation loss migliore a fini di analisi, ma non implementa early stopping né ripristina un checkpoint. I pesi finali addestrati sono quelli dopo l'epoca 100.

Probabilità apprese

Dopo il training, una riga della matrice dei pesi finale può ancora essere interpretata come distribuzione del carattere successivo per un determinato contesto.

Per il carattere m, le probabilità più alte sono:

o: 0.2497 e: 0.1818 a: 0.1810
s: 0.1126 p: 0.1120 spazio: 0.0427

Questi valori sono appresi soltanto dal sottoinsieme di training, quindi non devono necessariamente coincidere con le proporzioni empiriche dell'intero corpus o con il risultato full-batch della Versione 2.

Generazione neurale del testo

La generazione conserva lo stesso ciclo autoregressivo della Versione 2:

- converte il carattere corrente nel relativo ID intero;
- seleziona i logits addestrati corrispondenti e applica softmax;
- campiona il prossimo ID dalla distribuzione di probabilità;
- riconverte l'ID campionato in un carattere;
- usa quel carattere come contesto successivo.

La dimostrazione genera 300 caratteri usando i pesi finali dell'epoca 100. Un seed locale rende il risultato riproducibile.

Controlli inclusi

Le asserzioni del notebook verificano numero degli esempi, forme one-hot e dei parametri, split completo e senza sovrapposizioni, lunghezza delle storie di loss, valori finiti, riduzione della training loss, normalizzazione delle probabilità, ripetibilità del training, generazione riproducibile e lunghezza dell'output richiesta.

Un'esecuzione completa dall'alto verso il basso termina con il messaggio All checks passed.

Cosa insegna la Versione 3

La progressione centrale è:

`split -> shuffle -> batch -> update -> validate -> reproduce -> sample`

La Versione 3 separa l'architettura del modello dalla procedura di training. Il modello predittivo resta volutamente invariato, così che l'attenzione didattica rimanga sull'organizzazione e sulla valutazione dell'esperimento.

Interpretazione

Il training set aggiorna i parametri; il validation set misura lo stesso obiettivo di previsione del carattere successivo su esempi mantenuti separati per indice. Monitorare entrambe le loss rende visibile la differenza tra comportamento di ottimizzazione e comportamento in validation.

La validation loss minima all'epoca 64 è un risultato diagnostico osservato, non un checkpoint selezionato. La generazione del testo usa comunque i parametri finali dell'epoca 100.

Limitazioni

- il contesto contiene un solo carattere;
- il corpus è piccolo e incorporato nel notebook;
- lo split casuale a livello di esempio può collocare transizioni di caratteri ripetute sia nel training sia nella validation;
- non sono presenti early stopping, ripristino di checkpoint, hidden layer, embedding, differenziazione automatica o framework;
- la validation monitora esempi separati ma non costituisce una stima robusta della generalizzazione a un corpus differente;
- il testo generato conserva regolarità locali ma non coerenza a lungo termine.

Conclusione

La Versione 3 trasforma il modello neurale a caratteri in NumPy in un esperimento di training riproducibile con split training/validation, mini-batch, monitoraggio per epoca e riesecuzioni deterministiche. Prepara il progetto alla Versione 4, nella quale lo stesso piccolo modello potrà essere ricostruito con TensorFlow e differenziazione automatica dopo averne già compreso i calcoli fondamentali.